

Reference Notes

Week 8: List Mutability and Memory

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

Why do some data types behave differently when we try to change them? This week, we dive deep into one of Python's most important concepts: how lists actually work in memory. Understanding this will prevent countless bugs in your future programs and help you write more efficient code.

Prerequisites

This week builds on Week 7's list basics. You should be comfortable with:

- Creating and modifying lists
- Using list methods like `append()`, `remove()`, and `pop()`
- Accessing list elements with indexing

1.2 What You Already Know

From previous weeks, you've learned that strings are immutable - they cannot be changed after creation. When you "modify" a string, Python actually creates a new string. You've also started working with lists and know they can be changed. Now we'll explore exactly what this means and why it matters.

1.3 What You'll Be Able to Do

By the end of this week, you'll understand:

- Why changing one list sometimes affects another
- How to visualize list behavior using memory diagrams
- The difference between copying a reference and copying data
- How to avoid common bugs caused by aliasing
- When and how to create independent copies of lists

2 VIDEO 1: List Mutability - What Makes Lists Different

2.1 Understanding Mutability

Unlike strings, lists are mutable - they can be changed "in place" without creating a new object. This is powerful but can lead to surprising behavior if you don't understand what's happening in memory. When you modify a list, you're changing the actual data structure that already exists, not creating a new one.

2.2 Seeing Mutability In Action

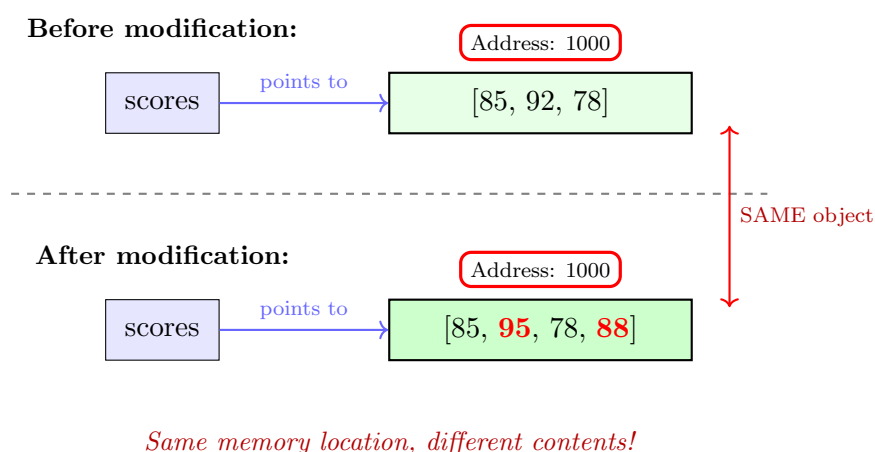
Let's start with a simple example that shows the fundamental difference between strings and lists. Watch how the list changes while maintaining the same identity, unlike a string which would create a new object.

```
1 # Lists can be modified in place
2 scores = [85, 92, 78]
3 print("Original:", scores)
4 print("ID:", id(scores))
5
6 scores[1] = 95 # Change second score
7 scores.append(88) # Add new score
8 print("Modified:", scores)
9 print("ID:", id(scores)) # Same ID!
```

Notice how the ID (memory address) stays the same even after we modify the list. This is mutability - the list object itself is being changed, not replaced with a new one.

2.3 Visualizing Mutability

To truly understand mutability, we need to see what's happening in memory. This diagram shows how the list object stays in the same memory location while its contents change, which is fundamentally different from how strings behave.



The key insight here is that the list object at memory address 1000 has been modified directly. We didn't create a new list - we changed the existing one. This is what makes lists mutable.

2.4 Comparing with Immutable Strings

To appreciate what mutability means, let's contrast it with strings. This comparison will help you understand when Python creates new objects versus modifying existing ones.

```

1 # Strings create new objects
2 name = "Ahmed"
3 print("Original ID:", id(name))
4
5 name = name + " Khan" # Creates NEW string
6 print("New ID:", id(name)) # Different ID!
7
8 # Lists modify in place
9 cities = ["Karachi", "Lahore"]
10 print("Original ID:", id(cities))
11
12 cities.append("Islamabad") # Modifies SAME list
13 print("Same ID:", id(cities)) # Same ID!

```

The string operation creates an entirely new string object, while the list operation modifies the existing list. This distinction becomes crucial when we have multiple variables referring to the same data.

⚠ Common Error

Common Mistake: Thinking that `list[0] = value` creates a new list. Remember: Modifying a list element changes the existing list, it doesn't create a new one!

👉 Quick Check

What will this print?

```

1 nums = [10, 20, 30]
2 original_id = id(nums)
3 nums[1] = 999
4 print(id(nums) == original_id)

```

Think about it, then test it!

3 VIDEO 2: References and Assignment - How Python Handles Lists

3.1 Understanding References

When you assign a list to a variable, Python doesn't copy the list data. Instead, it creates a reference - think of it as an arrow pointing to where the list lives in memory. This is efficient but can lead to unexpected behavior if you're not careful about how assignment works with mutable objects.

3.2 Assignment Creates References

Let's see what really happens when we assign one list variable to another. This is where many programmers encounter their first major confusion with lists.

```

1 # Assignment creates a reference, not a copy
2 original = [1, 2, 3]
3 reference = original # NOT a copy!
4

```

```

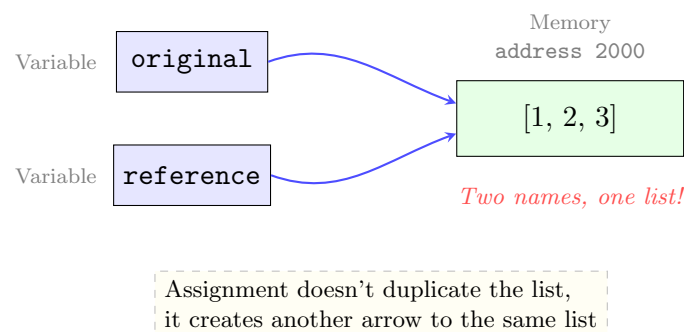
5 print("Original:", original)
6 print("Reference:", reference)
7 print("Same object?", original is reference)  # True

```

The `is` operator checks if two variables refer to the exact same object in memory. When it returns `True`, we know both variables point to the same list, not two separate lists with the same values.

3.3 Visualizing References

Understanding references is crucial for working with lists effectively. This memory diagram shows exactly what happens during assignment and why both variables end up pointing to the same list object.



This diagram reveals the truth: `reference = original` doesn't create a new list. It creates a new variable that points to the same list object. This is why Python is efficient with memory - it doesn't duplicate data unnecessarily.

3.4 The Consequence of References

Since both variables point to the same list, modifying through one variable affects what we see through the other. This behavior surprises many new programmers but makes perfect sense once you understand references.

```

1 # Modifying through one reference affects both
2 original = [1, 2, 3]
3 reference = original
4
5 reference.append(4)  # Add to 'reference'
6 print("Original:", original)  # [1, 2, 3, 4]
7 print("Reference:", reference)  # [1, 2, 3, 4]
8
9 original[0] = 99  # Modify through 'original'
10 print("Both show:", original)  # [99, 2, 3, 4]

```

Both variables show the changes because they're looking at the same list object in memory. This isn't a bug - it's how references work! Understanding this helps you write more predictable code.

💡 Rule

Assignment with lists creates a reference to the same object, not a copy of the data. Multiple variables can refer to the same list.

Quick Check

Predict the output:

```
1 a = [10, 20]
2 b = a
3 b[0] = 99
4 print(a)
```

Draw a memory diagram before checking your answer!

4 VIDEO 3: Aliasing - When Lists Share Memory

4.1 Understanding Aliasing

Aliasing occurs when two or more variables refer to the same object in memory. With lists, this means multiple "names" for the same data. While aliasing can be useful for efficiency, it often leads to bugs when programmers don't realize their variables are connected.

4.2 Aliasing in Real Programs

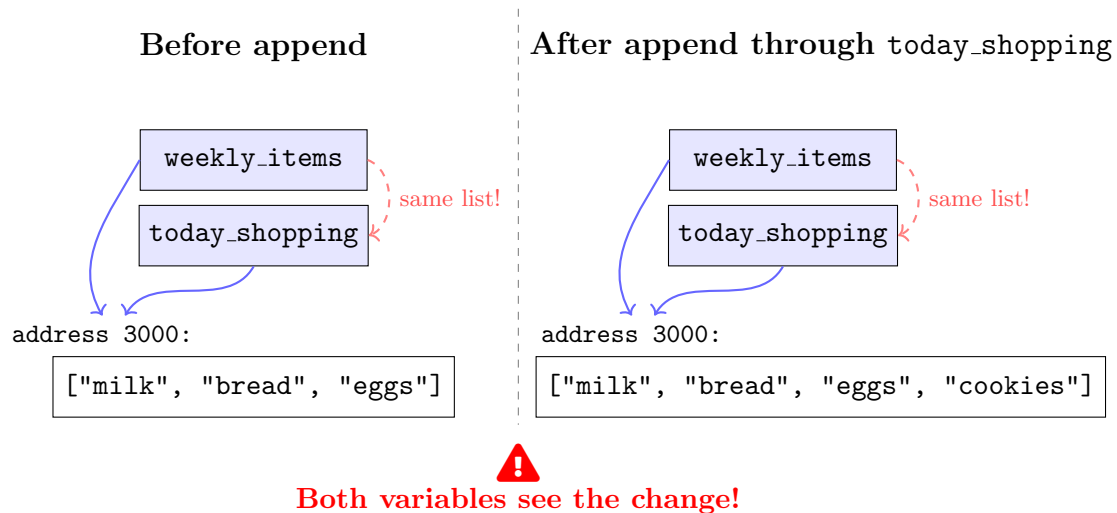
Let's look at a practical example where aliasing might occur naturally in a program. This shopping list scenario shows how aliasing can happen without you explicitly planning for it.

```
1 # A shopping list program with aliasing
2 weekly_items = ["milk", "bread", "eggs"]
3 today_shopping = weekly_items # Aliasing!
4
5 # Later in the program...
6 today_shopping.append("cookies")
7 print("Today:", today_shopping)
8 print("Weekly:", weekly_items) # Surprise!
9 # Both show: ["milk", "bread", "eggs", "cookies"]
```

You might have intended `today_shopping` to be a separate list, but instead you created an alias. Now both variables refer to the same list, and changes through either variable affect both.

4.3 Visualizing Aliasing Effects

This diagram shows how aliasing creates multiple pathways to the same data. Understanding this visual representation helps you recognize and debug aliasing issues in your programs.



Aliasing means multiple names for the same object

The diagram makes it clear: when you modify the list through `today_shopping`, you're modifying the only list that exists. Since `weekly_items` points to that same list, it sees the changes too.

4.4 Aliasing with Function Arguments

Aliasing becomes especially tricky when passing lists to functions. When you pass a list to a function, you're passing a reference, which can lead to unexpected modifications of your original data.

```

1 def add_bonus(scores):
2     scores.append(5) # Modifies the original!
3     return scores
4
5 test_scores = [85, 90, 78]
6 updated = add_bonus(test_scores)
7
8 print("Test scores:", test_scores) # [85, 90, 78, 5]
9 print("Updated:", updated) # [85, 90, 78, 5]
10 print("Same list?", test_scores is updated) # True

```

The function receives a reference to the original list, not a copy. Any modifications inside the function affect the original list. This is powerful but requires careful consideration in your program design.

⚠ Common Error

Common Mistake: Thinking that `list2 = list1` creates a copy. This creates an alias (another reference to the same list), not an independent copy!

💡 Tip

To check if two variables refer to the same object, use the `is` operator. To check if two lists have the same values, use `==`.

Quick Check

Will this cause aliasing? Think before testing:

```
1 a = [1, 2, 3]
2 b = [1, 2, 3]
3 print(a is b)
4 print(a == b)
```

5 VIDEO 4: Creating True Copies

5.1 Understanding the Need for Copies

Sometimes you need an independent copy of a list - one that you can modify without affecting the original. Python provides several ways to create copies, and choosing the right method depends on what your list contains.

5.2 Three Ways to Copy

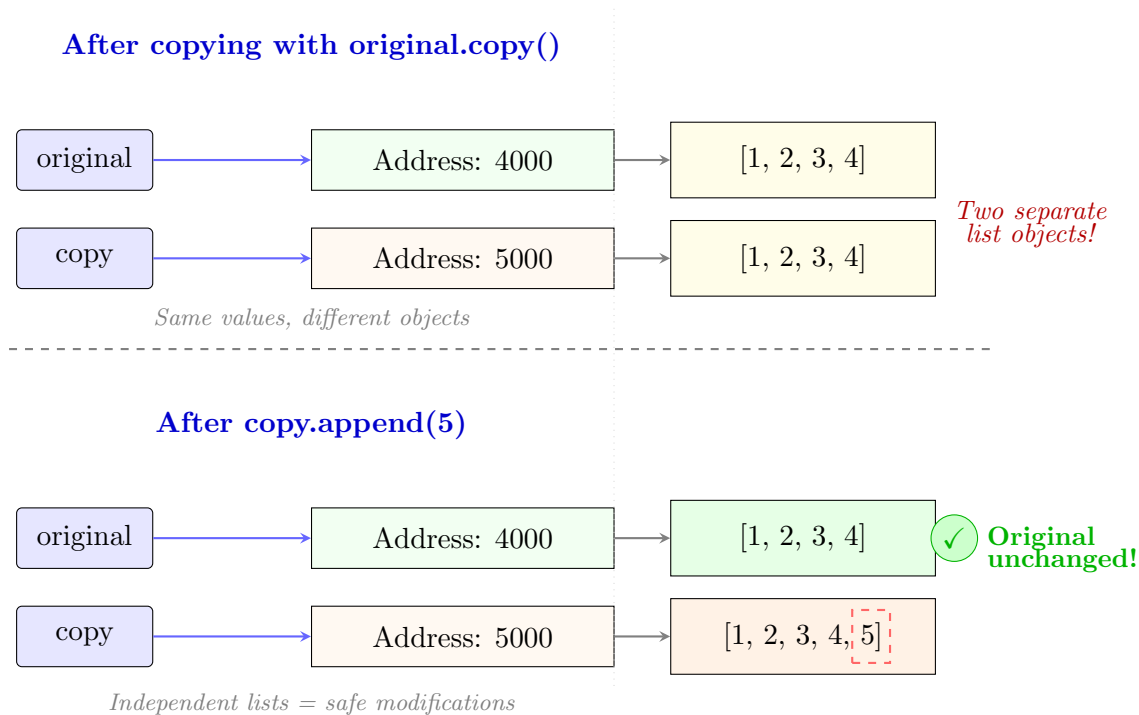
Python offers multiple approaches to copying lists. Let's explore each method and understand when to use them. Each has its place in different programming scenarios.

```
1 original = [1, 2, 3, 4]
2
3 # Method 1: Slice notation
4 copy1 = original[:]
5
6 # Method 2: list() constructor
7 copy2 = list(original)
8
9 # Method 3: copy() method
10 copy3 = original.copy()
11
12 # All create independent copies
13 print(original is copy1) # False
14 print(original is copy2) # False
15 print(original is copy3) # False
```

Each method creates a new list object with the same values. These are independent copies - modifying one won't affect the others. This is what you need when you want to work with list data without side effects.

5.3 Visualizing True Copies

Let's see how copying creates completely separate list objects in memory. This visualization shows why modifications to a copy don't affect the original - they're truly independent.



This diagram demonstrates the key difference: copying creates a new list object at a different memory address. Changes to the copy don't affect the original because they're completely separate objects.

5.4 Testing Copy Independence

Let's verify that our copies are truly independent by modifying them and checking the original remains unchanged. This proof helps solidify your understanding of the difference between aliasing and copying.

```

1 # Prove copies are independent
2 original = ["apple", "banana"]
3 safe_copy = original.copy()
4
5 safe_copy.append("orange")
6 safe_copy[0] = "mango"
7
8 print("Original:", original) # ["apple", "banana"]
9 print("Copy:", safe_copy)   # ["mango", "banana", "orange"]
10 # Original is protected!

```

The original list remains exactly as it was, while the copy has been modified. This independence is exactly what we want when we need to preserve original data while working with a modified version.

5.5 Comparing Copy Methods

While all three methods create shallow copies, there are subtle differences worth understanding. This comparison helps you choose the most appropriate method for your situation.

Method	Best Use Case	Readability
<code>original[:]</code>	Quick copies in algorithms	Less obvious
<code>list(original)</code>	Converting other sequences	Clear intent
<code>original.copy()</code>	Standard list copying	Most explicit

The `.copy()` method is generally preferred because it clearly communicates your intention to create a copy. However, you'll see all three methods in real Python code, so it's important to recognize them all.

💡 Rule

To create an independent copy of a list, use `.copy()`, `list()`, or slice notation `[:]`. Never use simple assignment if you need a copy.

👉 Quick Check

Fix this code so it doesn't have aliasing:

```
1 grades = [85, 90, 78]
2 backup = grades # Problem here!
3 grades[0] = 100
4 # How can we protect backup?
```

6 VIDEO 5: Summary

6.1 What We Learned This Week

This week, you've mastered one of Python's most important concepts: how lists behave in memory. You now understand that lists are mutable objects that can be modified in place, and that assignment creates references rather than copies. You've learned to recognize aliasing - when multiple variables refer to the same list - and how to create independent copies when needed. These concepts will help you write more predictable programs and debug mysterious behavior in your code.

You can now:

- Explain why lists are mutable and what this means in practice
- Draw memory diagrams showing how lists and references work
- Predict when aliasing will occur and its effects on your program
- Create true copies of lists to avoid unwanted side effects
- Debug programs where unexpected list modifications occur

★ Landmark Moment

You now understand one of the most important distinctions in programming: the difference between a reference and a copy. This knowledge will serve you well in any programming language you learn!

7 Quick Reference

Quick Reference

Week 8 Essential Patterns:

List Mutability:

- Pattern: `list[index] = value` modifies in place
- Memory: Original list object is changed
- Common mistake: Expecting a new list to be created

References and Assignment:

- Pattern: `list2 = list1` creates a reference
- Memory: Both variables point to same list
- Debug tip: Use `is` to check if same object

Creating Copies:

- Pattern: `list2 = list1.copy()` creates independent copy
- Memory: New list object at different address
- Common use: Protecting original data from modifications

Checking Identity:

- Pattern: `list1 is list2` checks same object
- Pattern: `list1 == list2` checks same values
- Debug tip: Different results indicate copy vs reference

7.1 Reflection Questions

1. What was the most surprising thing you learned about how lists work in memory?
2. Can you think of a time when aliasing might be useful rather than problematic?
3. How would you explain the difference between `=` and `.copy()` to a friend?
4. What debugging strategies will you use when you suspect aliasing issues?
5. How does understanding memory diagrams help you predict program behavior?

Next week: We'll explore advanced list algorithms including searching and sorting techniques!